arXiv:2512.12688v2 [cs.LG] 13 Jan 2026

Theoretical Foundations of Prompt Engineering: From Heuristics to Expressivity

Dongseok Kim [*1] **Hyoungsun Choi** [*1] **Mohamed Jismy Aashik Rasool** [*1] **Gisung Oh** [1]

Abstract

Prompts can switch a model's behavior even when the weights are fixed, yet this phenomenon is rarely treated as a clean theoretical object rather than a heuristic. We study the family of functions obtainable by holding a Transformer backbone fixed as an executor and varying only the prompt. Our core idea is to view the prompt as an externally injected program and to construct a simplified Transformer that interprets it to implement different computations. The construction exposes a mechanism-level decomposition: attention performs selective routing from prompt memory, the FFN performs local arithmetic conditioned on retrieved fragments, and depth-wise stacking composes these local updates into a multi-step computation. Under this viewpoint, we prove a constructive existential result showing that a single fixed backbone can approximate a broad class of target behaviors via prompts alone. The framework provides a unified starting point for formalizing trade-offs under prompt length/precision constraints and for studying structural limits of prompt-based switching, while remaining distinct from empirical claims about pretrained LLMs.

1. Introduction

Prompts often function like a switch: with the same underlying model, changing only the prompt can effectively change the task. In practice, modifying the prompt alone can drastically alter the output format, the style of reasoning, and even characteristic failure modes. Yet prompts are still largely discussed at the level of "tips and tricks," and it remains difficult to state mathematically why such switching is possible, how far it can go, and when or why it breaks. This gap has made it hard to systematize prompt engineering as a theoretical subject, despite its practical impact.

This paper reframes the question from "what can a model do?" to "what can a prompt change in a fixed model?" The core idea is to view a prompt not merely as an instruction string but as an external parameter (a program input) interpreted by a fixed Transformer backbone. This viewpoint lets us treat, as a single object, the range of behaviors achievable by varying prompts, and to restate "why/how far/when does it fail?" in the language of expressivity and structural constraints.

Building on this perspective, we prove a foundational possibility result: there exists a single fixed Transformer that, with suitably chosen prompts, can approximate any continuous target behavior on a compact input domain to arbitrarily small error. Importantly, we do not leave this as a purely declarative statement. We give a constructive account of the internal roles that enable switching and make them reusable as lemmas: attention provides selective *routing* from prompt memory, the FFN provides local *arithmetic* using retrieved fragments, and depth-wise repetition provides *composition* that accumulates multi-step computation across blocks. Our contribution is not an empirical claim about pretrained LLMs, nor an algorithm for finding effective prompts under realistic constraints; rather, it provides a precise framework that makes such follow-up questions well-posed, including trade-offs under length and precision constraints, constraint-driven limits, and learnability or search procedures for finding good prompts.

Contributions.

1. **Prompts as a theoretical object.** We formalize the prompt-induced behaviors of a fixed model as a single object, enabling an expressivity-based view of prompt design.

2. **An existence result for fixed-backbone prompt switching.** We show that a single fixed Transformer can approximate a broad target class by changing only the prompt, without task-specific weight updates.

3. **A structural explanation of why switching works.** We explain switching via a lemma-based decomposition into routing (selective readout), arithmetic (local computation), and composition (multi-step accumulation) across blocks.

[*]Equal contribution [1]Department of Computer Engineering, Gachon University, Seongnam, Gyeonggi, Republic of Korea. Correspondence to: Gisung Oh <eustia@gachon.ac.kr>.

Preprint. January 14, 2026.

4. **A roadmap for quantitative and algorithmic questions.** We identify principled directions including length/precision/norm trade-offs, limits for finite hard prompts, constraints induced by pretraining and architecture, and the learnability of effective prompts under practical optimization.

2. Related Work

2.1. Universal Approximation and Expressivity Foundations

Universal approximation characterizes expressivity via density of realizable functions in spaces such as $C(K)$ for compact K. Recent theory refines the classical UAT by providing quantitative rates tied to architectural capacity and function-space regularity (e.g., Besov-type smoothness for ReLU networks) (Siegel, 2023), and by identifying minimal width conditions and sharp thresholds under which universality holds or fails even on compact domains (Kim et al., 2023). Extensions to broader settings consider uniform approximation beyond compact domains, where tail control becomes essential (Bournez et al., 2025), and alternative parameterizations that preserve approximation goals while altering the mechanism (Liu et al., 2024). Related ideas appear in operator learning, which elevates universality to maps between function spaces and connects approximation theory to PDE solution operators (Kovachki et al., 2023; Furuya et al., 2023; 2024b). Our work follows this modern viewpoint but changes the quantifier structure: rather than a weight-indexed family across tasks, we study the prompt-indexed hypothesis class induced by a *fixed* executor, asking when varying only the prompt yields density in $C(K)$ and what structural constraints govern such switching.

2.2. Expressivity and Approximation Results for Transformers

A growing theory treats Transformers as explicit function approximators and derives approximation guarantees under capacity and regularity assumptions (Takakura & Suzuki, 2023; Jiang & Li, 2024). Closely related are results on prompt- or context-conditioned families, where context acts as a control signal that expands the realizable set even with constrained core weights (Petrov et al., 2024; Furuya et al., 2024a). Complementary complexity- and mechanism-oriented analyses link Transformer expressivity to structured computation along the sequence and identify intrinsic limitations tied to width, depth, embedding dimension, and finite-precision or parallelizability constraints (Merrill & Sabharwal, 2023a; Sanford et al., 2023; Merrill & Sabharwal, 2023b). Our setting aligns with these perspectives but focuses on a single fixed executor with prompt-controlled computation, made explicit via a routing/arithmetic/composition decomposition.

2.3. Prompting as Adaptation: Soft Prompts, Prefix Tuning, and Prompt Tuning

Prompt-based adaptation is a widely used parameter-efficient alternative to full fine-tuning, and many methods can be viewed as updating a small task-specific parameterization while keeping most weights fixed (Ding et al., 2023). Recent variants design prompts to steer attention patterns (Wang et al., 2023), construct compositional or codebook-based prompts to control parameter growth (Lin et al., 2024), and stabilize prompt reliance to improve robustness and transfer (Li et al., 2024). Other work connects prompt tuning to low-rank reparameterizations and hybrid prompt modules that improve efficiency and flexibility (Jain et al., 2024; Tang et al., 2025), with analyses suggesting shared structure that benefits estimation and sample efficiency (Le et al., 2024). In contrast, we focus on expressivity: prompts index a hypothesis class under fixed weights, and we provide a constructive mechanism-level account of when prompt-only switching is possible in principle.

2.4. In-Context Learning and Prompting Theory

Theoretical work on in-context learning (ICL) interprets prompting as forward-pass adaptation and asks what algorithmic behavior is implemented when demonstrations are provided (Akyürek et al., 2022; Garg et al., 2022). Constructive and mechanistic analyses connect ICL to optimization dynamics and meta-learning views, showing that Transformer components can approximate gradient-based updates under suitable parameterizations (Von Oswald et al., 2023; Dai et al., 2023), and other perspectives emphasize compression into low-dimensional task representations (Hendel et al., 2023) or extend optimization views beyond linear settings (Cheng et al., 2023). Empirical and geometric probes further highlight robustness gaps and irregularities, motivating sharper conditions on what is learned in-context (Zhao et al., 2024). We are aligned with this agenda in treating prompting as computation, but we study an expressivity (existence) result for a fixed executor and make the switching mechanism explicit through routing/arithmetic/composition, rather than characterizing a particular learning rule or sample-efficiency regime.

2.5. Computational and Mechanistic Views: Attention as Retrieval and Transformers as Interpreters

Mechanistic-interpretability work increasingly models Transformers as decomposable computational systems, where attention behaves like structured routing and circuits implement recognizable subroutines on algorithmic tasks (Conmy et al., 2023; Nanda et al., 2023; Wang et al., 2022). In parallel, attention-as-retrieval abstractions connect factual recall and long-context behavior to controlled access to stored state or self-retrieval formalisms (Geva et al., 2023;

Xu et al., 2023; Rubin & Berant, 2024), and causal analyses of induction heads provide fine-grained evidence for routing-centric interpretations of ICL (Crosbie & Shutova, 2025). Our lemma-based decomposition can be seen as a proof-level abstraction of these roles: prompt slots act as addressable memory, attention implements selective retrieval, the FFN performs local computation on retrieved fragments, and depth-wise composition accumulates these operations into a multi-step program.

3. Problem Setup

This section formalizes the approximation problem under the "fixed-backbone + prompt" viewpoint. The key difference from the classical UAT, which changes weights across tasks, is that we ask whether a *single fixed executor (backbone)* can *switch* among different functions via prompts. To this end, we define the prompt-induced family of functions and take its density in $(C(\mathcal{X}), \|\cdot\|_\infty)$ as our main objective.

3.1. Input domain and target function class

Assumption 3.1 (Input domain)**.** The input space is $\mathcal{X} \subset \mathbb{R}^d$, where $\mathcal{X}$ is compact.

Let $C(\mathcal{X})$ denote the space of real-valued continuous functions on $\mathcal{X}$, and measure approximation error in the sup-norm:

$$\|f - g\|_\infty := \sup_{x\in\mathcal{X}} |f(x) - g(x)|.$$

Thus, for any $g \in C(\mathcal{X})$ and $\varepsilon > 0$, the essence of the problem is whether the chosen model class can provide a function f such that $\|f - g\|_\infty < \varepsilon$.

3.2. A fixed backbone and the prompt-induced hypothesis class

Definition 3.2 (Prompt space and prompt-conditional output)**.** Fix a maximum prompt length $L_{\max} \in \mathbb{N}$ and set $L := L_{\max}$. Let the space of length-L soft prompts (prefixes) be $\mathcal{P} := \mathbb{R}^{L\times d_p}$. For tasks requiring fewer than $L_{\max}$ slots, we use zero-padding (null slots). For fixed parameters θ^*, we denote the model's scalar output by

$$F_{\theta^*} : \mathcal{P} \times \mathcal{X} \to \mathbb{R}, \qquad (p, x) \mapsto F_{\theta^*}(p, x).$$

Assumption 3.3 (Well-definedness in $C(\mathcal{X})$)**.** For every prompt $p \in \mathcal{P}$, the map $x \mapsto F_{\theta^*}(p, x)$ is continuous on $\mathcal{X}$. Hence $\mathcal{H}_{\theta^*} \subset C(\mathcal{X})$ is well-defined.

Definition 3.4 (Prompt-induced hypothesis class)**.** We define the family of functions induced by a fixed backbone θ^* through prompts as

$$\mathcal{H}_{\theta^*} := \Big\{ F_{\theta^*}(p, \cdot) \ : \ p \in \mathcal{P} \Big\} \subset C(\mathcal{X}).$$

Definition 3.4 turns prompt engineering into a mathematical object. Prompt design can now be reduced to the question: "how far can $\mathcal{H}_{\theta^*}$ reach?"

3.3. The Prompt-UAT objective

Remark 3.5 (Unconstrained prompts). We allow the ambient prompt space $\mathcal{P} = \mathbb{R}^{L_{\max}\times d_p}$ without norm constraints to state a purely existential expressivity result; quantitative trade-offs (prompt norm/precision versus ε) are deferred. In the actual executor-level construction and error analysis, however, we work with an *admissible* subset $\mathcal{P}_{\mathrm{adm}} \subset \mathcal{P}$ produced by the encoding scheme, on which all stored records are uniformly bounded.

Clarification (why unconstrained prompts are not the punchline). Remark 3.5 should be read as an *upper-envelope* expressivity statement: we first establish what is possible in principle when the prompt is treated as an external continuous control input. In particular, Theorem 5.1 does *not* imply that finite-context LLMs can realize arbitrary tasks; rather, it motivates asking how the best achievable error scales under realistic constraints (finite L, finite precision, norm bounds). Accordingly, in Section 7 and Section 8 we make explicit where costs enter (memory size via L, routing reliability via Δ/τ, and arithmetic approximation via FFN width), and defer the resulting trade-offs to follow-up work.

Definition 3.6 (Prompt-UAT property)**.** We say that a fixed backbone θ^* satisfies *Prompt-UAT* if

$$\overline{\mathcal{H}_{\theta^*}}^{\|\cdot\|_\infty} = C(\mathcal{X}).$$

Equivalently, for every $g \in C(\mathcal{X})$ and $\varepsilon > 0$, there exists some $p_g \in \mathcal{P}$ such that $\|F_{\theta^*}(p_g, \cdot) - g\|_\infty < \varepsilon$.

Remark 3.7 (Scalar output w.l.o.g.). We state results for scalar outputs; vector-valued targets can be handled componentwise.

Remark 3.8 (Comparison to the classical UAT). The classical UAT chooses parameters θ_g (depending on g) and approximates via F_{θ_g}. In contrast, Prompt-UAT fixes a *single* θ^* and switches across targets g using only the prompt p. That is, the weights fix the executor, while the prompt specifies the task instance.

Theorem 3.9 (Prompt-UAT (informal statement))**.** *For an appropriately designed (sufficiently large) Transformer family, there exists a fixed parameter θ^* such that Definition 3.6 holds. In particular, for every $g \in C(\mathcal{X})$ and every $\varepsilon > 0$, there exists some $p_g \in \mathcal{P}$ satisfying*

$$\sup_{x\in\mathcal{X}} \big|F_{\theta^*}(p_g, x) - g(x)\big| < \varepsilon.$$

Remark 3.10 (Interpretation). Theorem 3.9 formalizes the view that "the prompt is a program and the fixed Transformer is an executor" via the density of $\mathcal{H}_{\theta^*}$ in $(C(\mathcal{X}), \|\cdot$